

JavaScript Notes - Modules & ES6 Features

1. Modules in JavaScript

Named Export

- You can have **multiple named exports** in one file
- The import name must **match** the export name
- You can rename during import using `as` keyword

Example:

```
// function.js
export function greet(name) {
  console.log("Welcome,", name);
}

export function sum(a, b) {
  return a + b;
}

// OR export together
function multiply(a, b) {
  return a * b;
}
export { multiply };
```

Import:

```
import { greet, sum, multiply } from "./function.js";
// Import with alias
import { sum as add } from "./function.js";
```

Default Export

- Only **one default export** per file
- Can be imported with **any name** you want

Example:

```
// function.js
export default function greet(name) {
  console.log("Welcome,", name);
}
```

Import:

```
import greet from "./function.js";  
// OR with any name  
import myGreet from "./function.js";
```

2. Array Destructuring

Extract values from arrays into separate variables.

Syntax:

```
let arr = [1, 2, 3, 4, 5, 6, 7];  
let [n1, n2, n3, ...rest] = arr;  
  
console.log(n1);    // 1  
console.log(n2);    // 2  
console.log(n3);    // 3  
console.log(rest);  // [4, 5, 6, 7]
```

Key Points:

- Variables get values in order
- Use `...rest` to collect remaining elements into an array
- `typeof rest` is `object` (arrays are objects in JS)

3. Object Destructuring

Extract properties from objects into variables.

Syntax:

```
let obj1 = {  
  id: 1,  
  ename: "Avinash"  
};  
  
let { ename } = obj1;  
console.log(ename); // "Avinash"  
  
// Trying to destructure non-existent property  
let { a } = obj1;  
console.log(a);    // undefined
```

Key Points:

- Variable name must **match** property name
- Non-existent properties return `undefined`

4. Spread Operator (...)

Expands arrays or objects.

Use Cases:

1. Copy/Merge Arrays

```
let arr2 = [100, 200, 300];
let arr3 = [...arr2, 400, 500, 600];
// arr3 = [100, 200, 300, 400, 500, 600]
```

2. Pass Array Elements as Arguments

```
let numbers = [1, 2, 3];
console.log(...numbers); // 1 2 3
```

5. Rest Parameter (...)

Collects multiple elements into an array.

Use Case: Function Parameters

```
function demo(n1, n2, n3, ...rest) {
  console.log(n1, n2, n3); // 10 20 30
  console.log(rest);      // [40, 50, 60]
  console.log(...rest);   // 40 50 60
}

demo(10, 20, 30, 40, 50, 60);
```

Key Points:

- Must be the **last parameter**
- Collects all remaining arguments
- `rest` is an array, `...rest` spreads the array

6. Rest vs Spread - Quick Difference

Feature	Rest (...)	Spread (...)
Purpose	Collects items into array	Expands array/object
Used in	Function parameters, destructuring	Function calls, array/object literals
Example	<code>function(...rest)</code>	<code>console.log(...arr)</code>

7. Using Modules in HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>JS Modules</title>
</head>
<body>
  <h1>JavaScript Modules</h1>

  <!-- Important: type="module" -->
  <script type="module" src="script.js"></script>
</body>
</html>
```

Key Point:

- Must use `type="module"` in script tag to use import/export
-

Interview Quick Tips

1. Named vs Default Export:

- Named: Multiple exports, exact name needed
- Default: One export, any name works

2. Destructuring:

- Arrays: Position matters `[a, b, c]`
- Objects: Name matters `{name, age}`

3. Rest/Spread:

- Same syntax `...`, different purpose
- Rest: Collects (left side)
- Spread: Expands (right side)

4. typeof Array:

- Always returns `"object"`
- Arrays are special objects in JavaScript